Name: _____
Surname: _____
Date: _____

Intructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

Circle the correct answer for each question, as each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Questions 7-9: Debugging (10 points each)

These exercises show code cells with one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

In these exercises, you will encounter problems that are more open-ended and may not have a single correct answer. Your task is to explain how you would approach each problem by breaking it down into smaller, manageable steps.
Describe how you would structure your code, which logical steps are needed, and what kind of Python tools or constructs (e.g., loops, conditionals, functions, lists, etc.) you would use. You are not required to write full code, but doing so is encouraged if it helps clarify your ideas.

---

1. After running this code, what will be the final value of the **variable "a"**?

```
1 a = 4 - 4 / 0
2 a = abs(a)
```

   (a) `a` = ∞
   (b) `a = 0`
   (c) `a` = $-\infty$
   (d) The code will raise an error.

2. Is there any difference between the commands in lines 2 and 3?

```
1 x = 3.14
2 y = str(x)  # Line 2
3 z = f"{x}"  # Line 3
```

   (a) Both commands produce the same output (meaning that `y = z`).
   (b) The commands produce a different output (meaning that `y` $\neq$ `z`).
   (c) Line 2 (y) raises an error.
   (d) Line 3 (z) raises an error.

3. Which of the following lines of code will **raise an error**?

```
sports = ("football", "chess", "dancing")
sports[1] = "basketball"        # Line 2
print(f"I love {sports[-1]}")   # Line 3
print(sports[0][4])             # Line 4
```

   (a) Line 2.

   (b) Line 3.

   (c) Line 4.

   (d) None of them.

4. Which message will **never be printed**, no matter the value of "name"?

```
name = "Daniel"  # Anything goes here

if name[0] == "d":
  print("Your name starts with d")
elif name[0].lower() == "d":
  print("Your name starts with D")
elif (type(name) == int) or (type(name) == float):
  print("A number is not a string!")
else:
  print("Your name starts with a letter other than d")
```

   (a) "Your name starts with D".

   (b) "A number is not a string!".

   (c) "Your name starts with a letter other than d".

   (d) All of them can be printed depending on "name".

5. After running this code, what will be the final value of the **variable "count_up"**?

```
count_up = 1

while count_up <= 4:
  count_up += count_up
```

   (a) `count_up = 4`

   (b) `count_up = 8`

   (c) The code will raise an error.

   (d) The code never stops.

6. After running this code, what will be the length of "ls_end"?

```
def fibonacci(n: int, ls: list=[1, 1]) -> list:
  # Add 'n' more values to the list using Fibonacci logic
  # where x_(i) = x_(i-2) + x_(i-1)
  for _ in range(n):
    val = ls[-2] + ls[-1]
    ls.append(val)
  return ls

ls_test1 = fibonacci(3)
ls_test2 = fibonacci(3, ls=[1, 1, 2])
ls_end = fibonacci(3)
```

(a) `len(ls_end) < 5`

(b) `len(ls_end) = 5`

(c) `len(ls_end) > 5`

(d) The code will raise an error.

Answers: 1-d, 2-a, 3-a, 4-b, 5-b, 6-c

7. The following Python code is intended to print the factorials of numbers starting from 3, incrementing by 1 each time, and continuing until the user decides to stop. However, the code contains **three errors** that prevent it from functioning correctly. Identify and fix these errors.

   - Line 4 raises the error `SyntaxError: invalid syntax`.
   - The loop should print "`3!=6`" but instead prints "`True`".
   - After fixing the previous errors, the loop should continue with "`4!=24`", then print "`5!=120`", etc., but instead it prints "`3!=18`", then "`3!=54`"...

```
1  # Initialize the loop control and factorial variables
2  number = 3
3  factorial = 6
4  continue = True
5
6  while continue:
7    factorial = factorial * number
8    print(f"{number!=factorial}")
9    ask = input("Continue? (yes/no): ")
10   # If the user says "yes", proceed to the next number
11   if ask == "yes":
12     continue = True
13   else:
14     continue = False
```

Answer:

There are three errors. The first one is in line 4: `continue` is a reserved keyword of Python (it skips to the next iteration of a loop), so it cannot be used as a variable name. This is what raises the `SyntaxError`. We fix it by choosing another name, for instance `keep_going`, in every line where it appears. *(+3 points)*

The second error is in the `print`. Inside the curly brackets of an f-string Python evaluates an expression, and `number!=factorial` is a comparison ("is `number` different from `factorial`?"), whose result is the boolean `True`. The `!=` must be outside the brackets, as text, and each variable inside its own brackets: `f"{number}!={factorial}"`. *(+3 points)*

The third error is the update of the variables. The code multiplies `factorial` by `number`, but `number` is never increased, so it is always multiplied by 3 ($6 \cdot 3 = 18$, $18 \cdot 3 = 54$...). In addition, the multiplication happens *before* the print, but the initial values `number = 3` and `factorial = 6` are already the first result to show. We fix it by printing first, and updating the values at the end of the loop, only when the user wants to continue: first `number += 1`, and then `factorial = factorial * number`. The order matters: the new factorial uses the new number. *(+4 points)*

(Optional, fixed code)

```
1  # Initialize the loop control and factorial variables
2  number = 3
3  factorial = 6
4  keep_going = True
5
6  while keep_going:
7      print(f"{number}!={factorial}")
```

```
    ask = input("Continue? (yes/no): ")
    # If the user says "yes", proceed to the next number
    if ask == "yes":
        number += 1
        factorial = factorial * number
    else:
        keep_going = False
```

8. The following code is intended to create two sets: one containing the original members of a group, and another containing both the original and new members. It then displays the difference between these sets. However, the output is not as expected. Explain the issue and fix the code.

```
1 group_before = {"Luke", "Han", "Leia"}
2 # Duplicate the original group and add the new members
3 group_now = group_before
4 for new_member in ["Ben", "Arturo"]:
5   group_now.add(new_member)
6
7 # Compare the groups
8 print(group_now - group_before)
9 # Expected output: {"Arturo", "Ben"}
10 # Actual output: set()  <--- Empty set
```

Answer:

The problem is in line 3, `group_now = group_before`. This does not create a copy of the set: it creates a second name for the **same** set. Sets are mutable, so when the loop adds the new members to `group_now`, it is also adding them to `group_before`. Both names always show the same members, and their difference is the empty set. *(+5 points)*

We need to create a real copy of the original set before modifying it, for instance with `group_before.copy()`. Using `set(group_before)` or `group_before.union(set())` is also valid. After that, the loop only modifies `group_now`, and the difference is {`"Arturo"`, `"Ben"`}. *(+5 points)*

(Optional, fixed code)

```
1 group_before = {"Luke", "Han", "Leia"}
2 # Duplicate the original group and add the new members
3 group_now = group_before.copy()
4 for new_member in ["Ben", "Arturo"]:
5     group_now.add(new_member)
6
7 # Compare the groups
8 print(group_now - group_before)
```

9. The function below is meant to count how many animals of each species are in a shelter, ignoring the differences between uppercase and lowercase. However, it raises a "`KeyError: 'Cat'`" on line 6. Identify and fix the issue(s).

```
1  def count_species(animals: list) -> dict:
2      """Count how many animals of each species are in the list,
3      ignoring uppercase and lowercase differences."""
4      counts = {}
5      for animal in animals:
6          counts[animal] += 1
7      return counts
8
9  print(count_species(["Cat", "dog", "cat", "Dog", "bird"]))
10 # Expected: {"cat": 2, "dog": 2, "bird": 1}
11 # Output: KeyError: 'Cat'
```

Answer:

The problem is line 6, which is the same as `counts[animal] = counts[animal] + 1`. The first time that a species appears, it is not yet a key of the dictionary, so Python cannot read its current value and raises the `KeyError`. *(+5 points)*

We need an if-clause inside the loop: if the species is already a key of the dictionary, we add 1 to its value, as we already do. If not, we create the key and initialize it to 1. *(+2 points)*

There is a second error, which we only see after fixing the first one. The code uses `animal` as the key, so `"Cat"` and `"cat"` are counted separately, but we want to ignore the differences between uppercase and lowercase. We need to convert the species to lowercase before using it as a key, with `animal.lower()`. *(+3 points)*

(Optional, fixed code)

```
1  def count_species(animals: list) -> dict:
2      counts = {}
3      for animal in animals:
4          species = animal.lower()
5          if species in counts:
6              counts[species] += 1
7          else:
8              counts[species] = 1
9      return counts
```

10. Write a function named `safe_divide(numerator, denominator)`. This function must be able to handle specific edge cases and return values in specific formats as described below:

    - If `denominator` is 0, the function should **print** the warning message `"Warning: Division by zero!"` and **return** 10^{10}.
    - If the result of the division is an integer (e.g., 10 divided by 5), the function should return an `int` value.
    - For all other cases, the function should return a `float` value representing the result of the division.

Answer:

The function receives two numbers, `numerator` and `denominator`. Because the output depends on the case, we will use an if-elif-else structure with three branches, one for each case. *(+2 points)*

Division by zero. The first branch must be the check `if denominator == 0`. We have to check it before dividing, because otherwise Python would raise a `ZeroDivisionError`. Inside it, we use `print("Warning: Division by zero!")`, and we return 10^{10}. Be careful: in Python the power is `10**10`, while `10^10` is a different operation (bitwise XOR) that returns 0. *(+3 points)*

Integer results. For the second case, we need to know if the division has no remainder. We can check it with the modulo operator, `numerator % denominator == 0`. Alternatively, we can compute the result first, and compare it against its integer version, `result == int(result)`. If it is true, we return an integer: `return int(numerator // denominator)`. Simply returning `numerator / denominator` is not enough, because the operator `/` always returns a float (`10 / 5` is `2.0`). *(+10 points)*

All other cases. We only get to the last branch (`else`) if the denominator is not 0 and the division is not exact, so we return `numerator / denominator`, which is a float. *(+5 points)*

Continue exercise 10

(Optional, code)

```python
def safe_divide(numerator, denominator):
    # Case 1: division by zero
    if denominator == 0:
        print("Warning: Division by zero!")
        return 10**10
    # Case 2: the division is exact, so we return an integer
    elif numerator % denominator == 0:
        return int(numerator // denominator)
    # Case 3: any other case, we return a float
    else:
        return numerator / denominator
```

11. You are building a fitness app. You are given a tuple of workout logs, each log consisting of a string with the heart rate readings (in beats per minute) separated by commas, for instance:

```
logs = (
    "120,125,130",  # Smooth
    "120,140,130",  # Not smooth (jump of 20)
    "100,110,121",  # Not smooth (jump of 11)
    "90",           # Smooth (a single reading has no jumps)
    "130,122,125",  # Smooth (decreases are allowed)
    "80,85,90,95,100,105,110"  # Smooth (any length)
)
```

A log is considered smooth if no two consecutive readings differ by more than 10 bpm, whether the heart rate goes up or down.

How would you write a function that, given **any** tuple of logs of **any size**, returns an integer: the number of smooth logs?

Answer:

As we need to check several logs inside a tuple, the best approach is to use a for loop. We look at one log at a time inside the loop, and apply the same rules to each of them: `for log in logs`. *(+5 points)*

Because we want to count the number of smooth logs, before the loop starts we initialize a count variable to 0, `count = 0`. Then, inside the loop, every smooth log increases that count by 1, with `count += 1`. At the end of the function, after the loop finishes, we return the count. *(+5 points)*

Now we move inside the for loop. We have a single log, which is a string. Because we want to compare its individual numbers, we turn that string into a list of numbers. For instance, the log `"120,125,130"` should be converted to `[120, 125, 130]`. We do this by chaining two operations. First, `ls_strings = log.split(",")` returns a list of strings, splitting the log at its commas. Next, we turn every string into a number with list comprehension and the `int()` function, `ls_nums = [int(s) for s in ls_strings]`. *(+5 points)*

Now that we have the list of numbers, we start another for loop, nested inside the logs loop: `for idx in range(1, len(ls_nums))`. Its goal is to compare every reading against the previous one. Before this nested loop, we initialize a variable `is_smooth = True`. Inside it, we use an if-clause to detect a jump that is too large. Because the heart rate can go up or down, we compare the absolute value of the difference, `abs(ls_nums[idx] - ls_nums[idx-1]) > 10`. If the clause is true, we set `is_smooth = False` and use `break`, as a single jump is enough to discard the log. After the nested loop, we check whether `is_smooth` is still `True`, and if so, we increase the count by 1. *(+5 points)*

Comment: A log with a single reading has no pairs to compare, so the nested loop does not run and the log remains smooth, as the exercise requires.

Continue exercise 11

(Optional, code)

```python
def exercise11(logs: tuple[str]) -> int:
    # Initialize the count of smooth logs
    count = 0
    # Loop through the tuple, looking at each log
    for log in logs:
        # Turn the string into a list of numbers
        ls_strings = log.split(",")  # example: ["120", "125", "130"]
        ls_nums = [int(s) for s in ls_strings]  # example: [120, 125, 130]
        # We assume the log is smooth at first
        is_smooth = True
        # Compare every reading against the previous one
        for idx in range(1, len(ls_nums)):
            if abs(ls_nums[idx] - ls_nums[idx-1]) > 10:
                # A single large jump is enough to discard the log
                is_smooth = False
                break
        # If the log has "survived" the test, count it
        if is_smooth:
            count += 1
    return count
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.